

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng bài toán Multi-Agent Pathfinding
trong game bắn xe tăng nhiều người chơi

DƯƠNG QUANG ĐÔNG

20225808

Chương trình đào tạo: Kỹ thuật phần mềm

Giảng viên hướng dẫn: ThS. Nguyễn Tiến Thành

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

NGUYỄN VĂN A

nguyenvanabc@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Chữ kí GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Bài toán tìm đường đa tác nhân (Multi-Agent Pathfinding – MAPF) yêu cầu lập kế hoạch đồng thời cho nhiều agent di chuyển từ vị trí xuất phát đến đích mà không va chạm nhau. Trong bối cảnh game chiến thuật thời gian thực, bài toán này càng trở nên khó khăn hơn khi môi trường thay đổi liên tục, đòi hỏi các agent phải tái lập kế hoạch nhanh trong vài mili-giây mỗi khung hình. Các giải pháp tìm đường truyền thống cho agent đơn lẻ như A* không đảm bảo tránh va chạm giữa nhiều agent, trong khi các phương pháp tối ưu toàn cục như CBS đòi hỏi chi phí tính toán quá lớn cho môi trường thời gian thực.

Đồ án này nghiên cứu và triển khai ba cấp độ thuật toán MAPF trong một game bắn xe tăng 2D nhiều người chơi xây dựng trên nền tảng Unity Engine: (1) A* đơn agent làm baseline, (2) PIBT (Priority Inheritance with Backtracking) triển khai trực tiếp trong Unity C# để phối hợp đa agent không va chạm, và (3) PIBT C++/TCP tích hợp máy chủ lập kế hoạch bên ngoài Unity qua kết nối TCP, mở rộng khả năng tận dụng thuật toán hiệu năng cao viết bằng C++. Game sử dụng các bản đồ chuẩn MovingAI MAPF benchmark, đảm bảo kết quả thực nghiệm có thể so sánh với nghiên cứu cộng đồng.

Đồ án đóng góp: (i) hệ thống đọc và dựng bản đồ động từ định dạng .map tại thời điểm chạy; (ii) triển khai A* và PIBT trong C# tích hợp hoàn toàn với vật lý Unity; (iii) giao thức TCP kết nối Unity và máy chủ PIBT C++; (iv) hệ thống backtest tự động với môi trường tĩnh và chương ngại động, xuất kết quả CSV để phân tích định lượng. Thực nghiệm trên 5 bản đồ với 3 thuật toán, 6 lần lặp mỗi tổ hợp (90 run mỗi bộ) cho thấy cả ba thuật toán đều hoàn thành nhiệm vụ trên 4/5 bản đồ; PIBT có số lần tái hoạch định cao hơn A* do cơ chế phối hợp đa agent; khi bật chương ngại động, số lần replan tăng trung bình 30–50% nhưng hệ thống vẫn duy trì được mục tiêu.

Sinh viên thực hiện
Dương Quang Đông

ABSTRACT

Multi-Agent Pathfinding (MAPF) requires simultaneous path planning for multiple agents navigating from their start positions to designated goals without collisions. In real-time strategy games, this problem becomes particularly challenging due to continuously changing environments, demanding agents to replan within milliseconds each frame. Traditional single-agent solutions such as A* do not inherently prevent inter-agent collisions, while globally optimal approaches like CBS impose computational costs too high for real-time environments.

This thesis studies and implements three MAPF algorithm tiers within a 2D multiplayer tank game built on Unity Engine: (1) single-agent A* as a baseline, (2) PIBT (Priority Inheritance with Backtracking) implemented directly in Unity C# for collision-free multi-agent coordination, and (3) PIBT C++/TCP integrating an external planning server over TCP, enabling the use of high-performance C++ algorithms alongside Unity’s physics. The game uses standard MovingAI MAPF benchmark maps, ensuring results are comparable with community research.

Key contributions include: (i) a runtime map loading system that constructs grid and physics colliders from the .map format without baking into the Unity scene; (ii) A* and PIBT implementations in C# fully integrated with Unity physics and the shared tank control pipeline; (iii) a TCP protocol bridging Unity and an external PIBT C++ server; and (iv) an automated backtest system supporting static and dynamic-obstacle environments, exporting per-run CSV metrics. Experiments across 5 maps, 3 algorithms, and 6 repetitions per combination (90 runs per condition) show all three algorithms successfully destroy the Eagle target on 4 of 5 maps; PIBT exhibits higher replanning counts than A* due to its cooperative coordination mechanism; and enabling dynamic obstacles increases replanning by 30–50% on average while the overall mission success rate remains stable.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài	2
1.3 Định hướng giải pháp.....	2
1.4 Bố cục đồ án	3
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU	4
2.1 Khảo sát hiện trạng	4
2.1.1 Các giải pháp điều hướng trong game hiện tại.....	4
2.1.2 Khảo sát bản đồ benchmark MAPF.....	5
2.2 Tổng quan chức năng.....	5
2.2.1 Biểu đồ use case tổng quan	5
2.2.2 Quy trình nghiệp vụ backtest.....	7
2.3 Đặc tả chức năng	7
2.3.1 Đặc tả UC1 – Chơi đơn một người	7
2.3.2 Đặc tả UC4 – Chạy backtest tự động.....	7
2.3.3 Đặc tả UC3 – Chọn thuật toán AI	7
2.4 Yêu cầu phi chức năng.....	7
CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG..	10
3.1 Bài toán MAPF	10
3.2 Thuật toán A*	10
3.3 Thuật toán PIBT	10
3.4 Unity Engine và C#	11
3.5 Giao tiếp TCP giữa Unity và máy chủ C++	11

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	12
4.1 Thiết kế kiến trúc.....	12
4.1.1 Lựa chọn kiến trúc phần mềm	12
4.1.2 Thiết kế tổng quan	12
4.2 Thiết kế chi tiết	12
4.2.1 Thiết kế giao diện	12
4.2.2 Thiết kế các lớp MAPF chính.....	13
4.2.3 Thiết kế cơ sở dữ liệu	13
4.2.4 Thiết kế luồng mạng nhiều người chơi	15
4.3 Xây dựng ứng dụng	15
4.3.1 Công cụ và thư viện sử dụng	15
4.3.2 Kết quả đạt được.....	15
4.3.3 Minh họa các chức năng chính	18
4.4 Kiểm thử và đánh giá.....	18
4.4.1 Cấu hình thực nghiệm	18
4.4.2 Kết quả thực nghiệm – Môi trường tĩnh	19
4.4.3 Kết quả thực nghiệm – Môi trường có chương ngại động.....	19
4.4.4 Nhận xét và đánh giá.....	21
4.5 Triển khai	22
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	23
5.1 Hệ thống đọc bản đồ động và dựng lưới thống nhất	23
5.2 Triển khai A* thời gian thực với tái hoạch định định kỳ	23
5.3 Tích hợp PIBT phối hợp đa agent không va chạm.....	24
5.4 Cầu nối TCP đến máy chủ PIBT C++	27
5.5 Hệ thống backtest tự động với chương ngại động	27

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	30
6.1 Kết luận.....	30
6.2 Hướng phát triển	30
TÀI LIỆU THAM KHẢO	32
PHỤ LỤC	34
A. KẾT QUẢ BACKTEST CHI TIẾT	34
A.1 Kết quả môi trường tĩnh	34
A.2 Kết quả môi trường có chương ngại động.....	34
B. ĐẶC TẢ USE CASE BỔ SUNG	36
B.1 Đặc tả UC2 – Chơi mạng LAN nhiều người	36
B.2 Đặc tả UC5 – Xem kết quả backtest.....	36

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ hoạt động kịch bản chơi đơn	6
Hình 2.2	Biểu đồ hoạt động kịch bản chơi mạng LAN	6
Hình 4.1	Biểu đồ gói UML của hệ thống Unity MAPF	13
Hình 4.2	Biểu đồ lớp các thành phần tìm đường và điều phối agent . .	14
Hình 4.3	Luồng dữ liệu MAPF: từ tệp bản đồ đến chuyển động agent	15
Hình 4.4	Biểu đồ trình tự tạo và tham gia phòng LAN	16
Hình 4.5	Biểu đồ trình tự sẵn sàng và bắt đầu ván chơi LAN	17
Hình 4.6	Quy trình thực nghiệm backtest tự động	19
Hình 4.7	Thời gian hoàn thành trung bình theo bản đồ (môi trường tĩnh)	20
Hình 4.8	Tổng số lần tái hoạch định trung bình theo bản đồ, thang log- arit (môi trường tĩnh)	20
Hình 4.9	So sánh số lần tái hoạch định giữa môi trường tĩnh và có chướng ngại động	21
Hình 5.1	Biểu đồ trình tự chu kỳ tái hoạch định của A*	25
Hình 5.2	Biểu đồ trình tự một tick lập kế hoạch của PIBT C#	26
Hình 5.3	Biểu đồ trình tự trao đổi thông điệp PIBT qua TCP	28

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các giải pháp điều hướng đa agent	5
Bảng 2.2	Năm bản đồ benchmark sử dụng trong đồ án	5
Bảng 2.3	Đặc tả UC1 – Chơi đơn	7
Bảng 2.4	Đặc tả UC4 – Chạy backtest tự động	8
Bảng 2.5	Đặc tả UC3 – Chọn thuật toán AI	8
Bảng 4.1	Công cụ và thư viện sử dụng	16
Bảng 4.2	Thống kê mã nguồn C# (tháng 6/2026)	17
Bảng 4.3	Cấu hình thực nghiệm	18
Bảng 4.4	Kết quả thực nghiệm môi trường tĩnh (trung bình 6 lần lặp, 6 agent)	19
Bảng 4.5	Kết quả thực nghiệm môi trường có chương ngại động (trung bình 6 lần lặp, 6 agent)	21
Bảng A.1	Kết quả backtest chi tiết – môi trường tĩnh ($n = 6$)	34
Bảng A.2	Kết quả backtest chi tiết – môi trường có chương ngại động ($n = 6$)	35
Bảng B.1	Đặc tả UC2 – Chơi mạng LAN	36
Bảng B.2	Đặc tả UC5 – Xem kết quả backtest	37

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
AI	Trí tuệ nhân tạo (Artificial Intelligence)
CBS	Tìm kiếm dựa trên xung đột (Conflict-Based Search)
CSV	Tập giá trị phân tách bằng dấu phẩy (Comma-Separated Values)
FPS	Số khung hình trên giây (Frames Per Second)
HUD	Bảng thông tin trên màn hình (Heads-Up Display)
LAN	Mạng cục bộ (Local Area Network)
LNS2	Tìm kiếm lân cận lớn lần 2 (Large Neighborhood Search 2)
LOS	Đường ngắm thẳng (Line of Sight)
MAPF	Tìm đường đa tác nhân (Multi-Agent Pathfinding)
PIBT	Kế thừa ưu tiên có hoàn tác (Priority Inheritance with Backtracking)
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
URP	Quy trình kết xuất đa năng của Unity (Universal Render Pipeline)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

Chương 1 giới thiệu bối cảnh, động lực và phạm vi của đề án. Từ bài toán điều hướng đa tác nhân trong game, chương này xác định mục tiêu cụ thể, đề xuất định hướng giải pháp và trình bày tổng quan cấu trúc báo cáo.

1.1 Đặt vấn đề

Trong các trò chơi điện tử chiến thuật thời gian thực (Real-Time Strategy – RTS), điều hướng đồng thời nhiều nhân vật AI đến mục tiêu là một thách thức kỹ thuật cốt lõi. Khi số lượng nhân vật AI tăng lên, xác suất xảy ra va chạm, tình trạng deadlock và đường đi không hiệu quả cũng tăng theo. Đây chính là bài toán MAPF – Multi-Agent Pathfinding – được nghiên cứu rộng rãi trong lĩnh vực trí tuệ nhân tạo và robot học [1].

MAPF yêu cầu lập kế hoạch đồng thời cho k agent trên đồ thị lưới để đảm bảo mỗi agent đến đích trong thời gian tối thiểu mà không va chạm nhau tại bất kỳ bước thời gian nào. Trong môi trường game thời gian thực, bài toán này đặt ra thêm ràng buộc về thời gian tính toán: thuật toán phải phản hồi trong vài mili-giây và phải tái lập kế hoạch liên tục khi môi trường thay đổi.

Các thuật toán tìm đường agent đơn như A* [2] được sử dụng phổ biến trong game nhờ độ đơn giản và hiệu quả, nhưng khi áp dụng độc lập cho nhiều agent, chúng không đảm bảo tránh va chạm. Các thuật toán MAPF tối ưu như CBS [3] đảm bảo tính tối ưu toàn cục nhưng có độ phức tạp tính toán tăng theo hàm mũ với số agent, không phù hợp cho game thời gian thực. PIBT (Priority Inheritance with Backtracking) [4] là một hướng tiếp cận cân bằng: chạy theo bước thời gian, hoàn chỉnh và hiệu quả cho môi trường thời gian thực, song không đảm bảo tối ưu toàn cục.

Thực tế trong ngành game, phần lớn các tựa game RTS và tower-defense hiện tại sử dụng thuật toán tìm đường đơn giản hoặc Unity NavMesh mà không giải quyết bài toán va chạm đa agent ở lớp lập kế hoạch. Điều này dẫn đến các hiện tượng nhân vật xếp hàng chờ hoặc chen lấn không tự nhiên, làm giảm chất lượng trải nghiệm người chơi.

Đề án này xuất phát từ nhu cầu nghiên cứu và so sánh thực nghiệm ba cấp độ thuật toán MAPF – từ baseline A* đến PIBT phối hợp đa agent và PIBT tích hợp máy chủ C++ bên ngoài – trong bối cảnh game bắn xe tăng 2D nhiều người chơi xây dựng trên Unity Engine. Kết quả sẽ cung cấp bằng chứng thực nghiệm về ưu nhược điểm của từng hướng tiếp cận trong môi trường game thực tế.

1.2 Mục tiêu và phạm vi đề tài

Các sản phẩm và nghiên cứu liên quan hiện tại có thể phân thành ba nhóm. Nhóm thứ nhất là các game thương mại sử dụng tìm đường đơn giản: phần lớn game RTS và tower-defense dùng A* hoặc Unity NavMesh cho từng agent độc lập, không giải quyết va chạm đa agent ở lớp lập kế hoạch, dẫn đến hiện tượng nhân vật xếp hàng hoặc chen lấn không tự nhiên. Nhóm thứ hai là các nghiên cứu MAPF học thuật như CBS, EECBS hay PIBT, được nghiên cứu sâu trong cộng đồng trí tuệ nhân tạo nhưng thường chỉ được đánh giá trên benchmark thuần túy mà không tích hợp trực tiếp vào engine game thương mại. Nhóm thứ ba là một số nghiên cứu thử tích hợp MAPF vào game [5], song thường chỉ chạy trên môi trường giả lập đơn giản, thiếu một engine vật lý đầy đủ.

Từ ba nhóm trên, có thể thấy khoảng trống hiện tại là thiếu một so sánh thực nghiệm đầy đủ giữa các cấp độ MAPF trong môi trường game có vật lý thực tế (Unity physics) trên tập bản đồ benchmark chuẩn. Đây chính là khoảng trống mà đề án hướng tới lấp đầy.

Trên cơ sở đó, đề án đặt ra mục tiêu xây dựng một game bắn xe tăng 2D nhiều người chơi trên Unity sử dụng bản đồ chuẩn MovingAI MAPF benchmark, đồng thời triển khai và tích hợp ba cấp độ thuật toán MAPF gồm A* làm baseline, PIBT C# và PIBT C++/TCP. Để so sánh ba thuật toán một cách khách quan, đề án xây dựng thêm một hệ thống backtest tự động đo lường định lượng trên năm bản đồ, từ đó phân tích kết quả và rút ra nhận xét về khả năng ứng dụng của từng thuật toán trong game thời gian thực.

Về phạm vi, đề án giới hạn ở game 2D góc nhìn từ trên xuống, môi trường lưới rời rạc (discrete grid), tối đa sáu agent AI trong kịch bản thực nghiệm, và không xét tối ưu hóa đa mục tiêu phức tạp.

1.3 Định hướng giải pháp

Từ phân tích ở Mục 1.2, đề án đề xuất ba bước triển khai:

Thứ nhất, xây dựng hạ tầng bản đồ thống nhất dựa trên đọc tệp .map MovingAI và dựng lưới ô tại thời điểm chạy trong Unity, thay vì bake cứng vào scene. Cách này cho phép thử nghiệm trên nhiều bản đồ mà không cần thay đổi code.

Thứ hai, triển khai tuần tự ba thuật toán MAPF chia sẻ cùng tầng điều khiển vật lý (TankController), đảm bảo hành vi vật lý nhất quán khi so sánh thuật toán. A* đóng vai trò baseline; PIBT C# thêm phối hợp đa agent ngay trong Unity; PIBT C++/TCP mở rộng tích hợp máy chủ ngoài.

Thứ ba, xây dựng hệ thống backtest tự động chạy kịch bản, thu thập chỉ số và

xuất CSV, cho phép so sánh định lượng trên tập bản đồ lớn mà không cần can thiệp thủ công. Thêm vào đó, module chương ngại động (spawn/despawn thùng cản đường trực tiếp trên path của agent) dùng để stress-test khả năng tái hoạch định của từng thuật toán.

Đóng góp chính: hệ thống tích hợp MAPF đầy đủ trong Unity với ba cấp độ thuật toán, cơ sở hạ tầng backtest định lượng, và bộ kết quả thực nghiệm trên benchmark chuẩn.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày khảo sát hiện trạng các giải pháp điều hướng đa agent trong game, phân tích yêu cầu chức năng và phi chức năng của hệ thống, đặc tả các use case chính thông qua biểu đồ use case và đặc tả luồng sự kiện.

Chương 3 trình bày nền tảng lý thuyết và công nghệ được sử dụng: bài toán MAPF và cách biểu diễn, thuật toán A* và heuristic Manhattan, thuật toán PIBT và cơ chế kế thừa ưu tiên có hoàn tác, cùng với lý do lựa chọn Unity Engine và giao thức TCP cho tích hợp máy chủ ngoài.

Chương 4 trình bày phân tích thiết kế kiến trúc hệ thống theo mô hình phân tầng, thiết kế chi tiết các lớp MAPF AI, quá trình xây dựng ứng dụng với thống kê mã nguồn, và đánh giá thực nghiệm với số liệu backtest trên 90 kịch bản tĩnh và 90 kịch bản có chương ngại động.

Chương 5 trình bày năm đóng góp kỹ thuật nổi bật: hệ thống đọc bản đồ động, triển khai A* thời gian thực, tích hợp PIBT phối hợp đa agent, cầu nối TCP đến máy chủ C++, và hệ thống backtest tự động với chương ngại động.

Chương 6 tổng kết kết quả đã đạt được, so sánh với mục tiêu đề ra, và định hướng phát triển trong tương lai.

Kết chương 1: Chương này đã xác định bài toán MAPF trong game thời gian thực, nêu rõ khoảng trống nghiên cứu và đề xuất hướng giải quyết bằng cách tích hợp và so sánh ba thuật toán (A*, PIBT C#, PIBT C++/TCP) trong môi trường Unity. Các chương tiếp theo sẽ trình bày chi tiết quá trình phân tích, thiết kế, triển khai và đánh giá hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Chương 1 đã xác định bài toán MAPF và đề xuất định hướng giải pháp. Chương này phân tích chi tiết hiện trạng các giải pháp điều hướng đa agent trong game, từ đó xác định yêu cầu chức năng và phi chức năng của hệ thống cần xây dựng.

2.1 Khảo sát hiện trạng

2.1.1 Các giải pháp điều hướng trong game hiện tại

Các giải pháp điều hướng agent trong game thương mại và nghiên cứu học thuật có thể phân thành bốn nhóm chính:

(1) Unity NavMesh: Unity Engine cung cấp hệ thống dẫn đường tự động dựa trên navigation mesh 3D. NavMesh tính toán đường đi cho từng agent độc lập và hỗ trợ tránh va chạm theo thời gian thực qua *NavMeshAgent*. Ưu điểm: tích hợp sẵn, dễ sử dụng, hỗ trợ môi trường 3D phức tạp. Nhược điểm: không giải quyết bài toán MAPF (không đảm bảo tránh va chạm ở lớp lập kế hoạch cho môi trường lưới rời rạc), không tương thích với benchmark .map chuẩn MAPF.

(2) A* đơn agent: Thuật toán A* [2] với heuristic Manhattan trên lưới ô vuông là giải pháp phổ biến nhất trong game 2D. Mỗi agent tìm đường độc lập mà không biết kế hoạch của agent khác. Ưu điểm: đơn giản, hiệu quả, dễ triển khai. Nhược điểm: va chạm giữa các agent phải xử lý ở lớp vật lý, không có phối hợp ở lớp lập kế hoạch, hiệu năng suy giảm khi số agent lớn do xử lý va chạm phản ứng.

(3) CBS và thuật toán MAPF tối ưu: Conflict-Based Search [3] đảm bảo giải pháp tối ưu toàn cục (tổng số bước di chuyển nhỏ nhất). Ưu điểm: tối ưu, hoàn chỉnh. Nhược điểm: độ phức tạp tính toán tăng theo hàm mũ với số agent và mật độ va chạm, không thực tế cho game thời gian thực với nhiều agent.

(4) PIBT: Priority Inheritance with Backtracking [4] là thuật toán MAPF chạy theo bước thời gian, hoàn chỉnh trên bản đồ có đường đi tồn tại, với độ phức tạp thực tế phù hợp cho môi trường thời gian thực. PIBT không đảm bảo tối ưu toàn cục nhưng thực hành cho kết quả tốt và đã được dùng trong các cuộc thi MAPF quốc tế [6].

Bảng 2.1 tổng hợp so sánh bốn giải pháp theo các tiêu chí phù hợp với yêu cầu của đề án.

Từ phân tích trên, đề án lựa chọn A* làm baseline để có điểm so sánh quen thuộc, và PIBT làm thuật toán chính vì cân bằng tốt giữa hiệu quả và khả năng thực thi thời gian thực.

Bảng 2.1: So sánh các giải pháp điều hướng đa agent

Tiêu chí	NavMesh	A* đơn	CBS	PIBT
Tránh va chạm đa agent	Vật lý	Vật lý	Lập kế hoạch	Lập kế hoạch
Tính tối ưu	Không	Không	Có	Không
Thời gian thực	Có	Có	Không	Có
Tương thích bản đồ .map	Không	Có	Có	Có
Tích hợp Unity	Có sẵn	Tự triển khai	Tự triển khai	Tự triển khai
Phù hợp đồ án	Không	Baseline	Không	Chính

2.1.2 Khảo sát bản đồ benchmark MAPF

MovingAI MAPF benchmark [7] cung cấp hàng nghìn bản đồ với định dạng .map chuẩn, phân loại theo môi trường (mê cung, trong nhà, game). Đồ án sử dụng 5 bản đồ đại diện cho 5 kịch bản điển hình (Bảng 2.2).

Bảng 2.2: Năm bản đồ benchmark sử dụng trong đồ án

Nhãn	Tệp bản đồ	Kích thước	Ô đi được	Đặc điểm
Alpha32	random-32-32-10.map	32×32	922 (90%)	Thoảng, ít tường
Mansion	ht_mansion_n.map	133×270	8959 (25%)	Hành lang hẹp
Chantry	ht_chantry.map	162×141	7461 (33%)	Phòng kín
Gallows	lt_gallowstemplar_n.map	251×180	10021 (22%)	Vùng hẹp nhiều
Maze128	maze-128-128-10.map	128×128	14818 (90%)	Mê cung lớn

2.2 Tổng quan chức năng

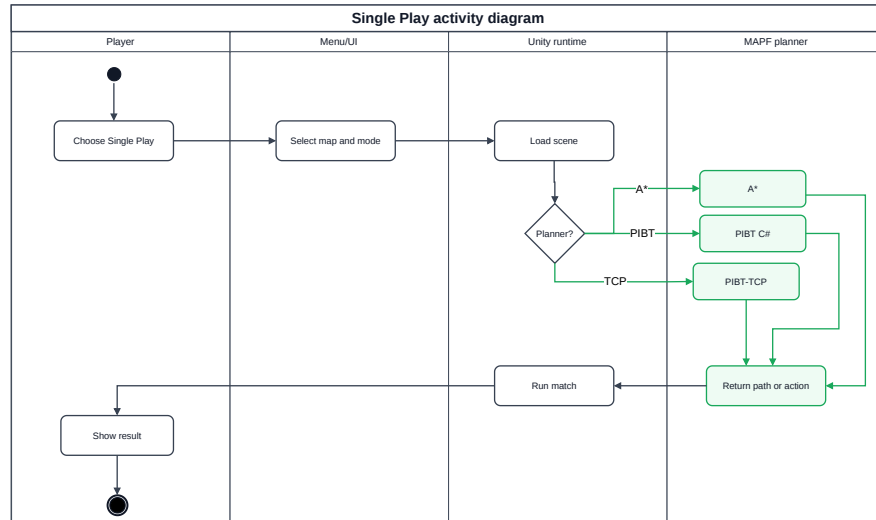
2.2.1 Biểu đồ use case tổng quan

Hệ thống có hai tác nhân chính. Tác nhân thứ nhất là người chơi (Player), trực tiếp điều khiển xe tăng, bắn đạn, chọn chế độ chơi, cũng như cấu hình và khởi động backtest. Tác nhân thứ hai là agent AI (Enemy), được điều khiển bởi thuật toán MAPF để tự động tìm đường và tấn công Eagle Base. Hai tác nhân này tương tác trong cùng một môi trường lưới, nơi người chơi cố gắng bảo vệ Eagle Base còn các agent AI phối hợp tấn công.

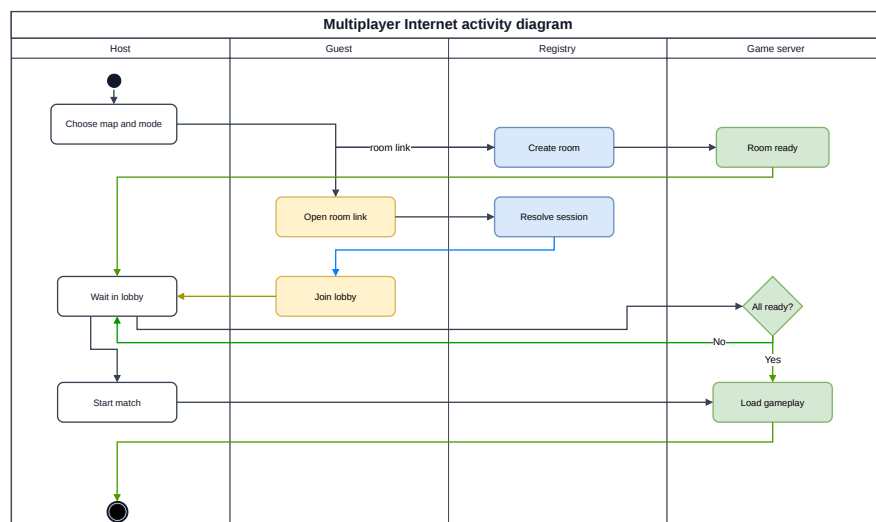
Các chức năng chính của hệ thống gồm: (UC1) Chơi đơn một người chống lại các agent AI, (UC2) chơi mạng LAN nhiều người cùng chống AI, (UC3) chọn thuật toán AI (A*, PIBT, PIBT TCP), (UC4) chạy backtest tự động, và (UC5) xem kết quả backtest. Trong đó UC1, UC3 và UC4 được đặc tả chi tiết tại Mục 2.3; UC2 và UC5 được đặc tả trong Phụ lục B.1 và B.2.

Hình 2.1 mô tả luồng hoạt động trong kịch bản chơi đơn, từ khi tải bản đồ đến khi kết thúc ván chơi.

Hình 2.2 mô tả luồng hoạt động trong kịch bản chơi mạng LAN, bao gồm bước tạo/tham gia phòng và đồng bộ trạng thái giữa các máy.



Hình 2.1: Biểu đồ hoạt động kịch bản chơi đơn



Hình 2.2: Biểu đồ hoạt động kịch bản chơi mạng LAN

2.2.2 Quy trình nghiệp vụ backtest

Backtest là quy trình nghiệp vụ quan trọng nhất từ góc độ nghiên cứu. Luồng hoạt động: người dùng cấu hình tham số (chọn bản đồ, thuật toán, số lần lặp, bật/tắt chướng ngại động), hệ thống tự động lặp qua tất cả tổ hợp, mỗi run thu thập chỉ số thời gian thực, sau đó xuất CSV và chart tổng hợp. Quy trình này được mô tả chi tiết trong Chương 4 tại Hình 4.6.

2.3 Đặc tả chức năng

2.3.1 Đặc tả UC1 – Chơi đơn một người

Bảng 2.3: Đặc tả UC1 – Chơi đơn

Tên use case	Chơi đơn (Single Play)
Tác nhân	Người chơi
Tiền điều kiện	Người chơi đã mở ứng dụng và chọn chế độ chơi đơn
Luồng chính	1. Người chơi chọn bản đồ và thuật toán AI từ menu. 2. Hệ thống đọc tệp .map, dựng bản đồ và spawn xe tăng người chơi + Eagle Base. 3. MapScenarioBootstrap spawn các agent AI theo thuật toán đã chọn. 4. Người chơi điều khiển xe tăng qua bàn phím/chuột; agent AI tự động di chuyển và tấn công Eagle. 5. Ván chơi kết thúc khi Eagle bị phá hủy (agent thắng) hoặc người chơi tiêu diệt hết agent.
Luồng phát sinh	2a. Tệp .map không tìm thấy → hệ thống dùng bản đồ mặc định Alpha32. 5a. Không có agent sống, Eagle còn HP → người chơi thắng.
Hậu điều kiện	Màn hình kết quả hiển thị điểm máu Eagle và thời gian ván chơi

2.3.2 Đặc tả UC4 – Chạy backtest tự động

2.3.3 Đặc tả UC3 – Chọn thuật toán AI

2.4 Yêu cầu phi chức năng

Hiệu năng: Thuật toán A* phải hoàn thành tìm đường cho mỗi agent trong dưới 5 ms trên bản đồ nhỏ (Alpha32), để tổng thời gian tính toán mỗi khung hình không vượt quá 16 ms (60 FPS). PIBT phải hoàn thành một bước phối hợp cho 6 agent trong dưới 10 ms.

Tính ổn định: Mỗi run backtest phải tái lập được với cùng seed; hệ thống không được crash trong suốt 90 run liên tiếp.

Tính tương thích: Ứng dụng chạy trên Windows 10/11 với Unity 6000.3.x; chế độ WebGL tương thích Chrome và Firefox. Tệp .map phải đọc được theo chuẩn MovingAI không cần chỉnh sửa.

Bảng 2.4: Đặc tả UC4 – Chạy backtest tự động

Tên use case	Chạy backtest tự động
Tác nhân	Người dùng (nghiên cứu viên)
Tiền điều kiện	Ứng dụng đang ở menu chính; nếu chọn PIBT_TCP, server C++ đang chạy
Luồng chính	1. Người dùng mở UI Backtest, chọn bản đồ, thuật toán, số lần lặp, bật/tắt chương ngại động. 2. Người dùng bấm Start. 3. BacktestRunner lặp qua từng tổ hợp (map × algorithm × rep). 4. Mỗi run: spawn agent, chạy kịch bản đến khi Eagle bị phá hoặc timeout, ghi nhận chỉ số. 5. Sau khi hết tất cả run: xuất backtest_summary_*.csv, backtest_agents_*.csv, backtest_chart_*.html.
Luồng phát sinh	3a. PIBT_TCP: nếu kết nối TCP thất bại, run đó ghi Outcome=ConnectionError và tiếp tục. 4a. Run chậm timeout (mặc định 120 s) → ghi Outcome=Timeout.
Hậu điều kiện	Tập CSV và HTML chart xuất hiện trong thư mục BacktestResults/

Bảng 2.5: Đặc tả UC3 – Chọn thuật toán AI

Tên use case	Chọn thuật toán AI
Tác nhân	Người chơi hoặc nghiên cứu viên
Tiền điều kiện	Ứng dụng đang ở menu chọn chế độ chơi
Luồng chính	1. Người dùng chọn một trong ba thuật toán: A*, PIBT, PIBT_TCP. 2. Hệ thống ghi nhận lựa chọn vào BacktestMode.Algorithm. 3. Khi bắt đầu ván: MapScenarioBootstrap spawn đúng loại agent tương ứng (GridEnemyAgent, GridEnemyAgentPIBT, hoặc GridEnemyAgentPIBT_TCP). 4. Nếu chọn PIBT_TCP: hệ thống gửi lệnh Hello đến server, xác nhận kết nối trước khi spawn agent.
Luồng phát sinh	4a. Server C++ không phản hồi trong 5 s → báo lỗi và quay lại menu.
Hậu điều kiện	Agent AI trong kịch bản dùng thuật toán đã chọn

Khả năng mở rộng: Thêm thuật toán mới chỉ cần tạo lớp agent mới kế thừa interface chung, không cần thay đổi BacktestRunner hay TankController.

Kết chương 2: Chương này đã phân tích bốn nhóm giải pháp điều hướng đa agent và xác nhận A* (baseline) cùng PIBT (thuật toán chính) là lựa chọn phù hợp nhất với yêu cầu đề án. Năm bản đồ benchmark chuẩn đã được chọn để đảm bảo tính so sánh được với nghiên cứu cộng đồng. Các use case và yêu cầu phi chức năng được đặc tả là cơ sở cho thiết kế hệ thống trong Chương 3 và Chương 4.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương 2 đã xác định A* và PIBT là hai thuật toán cần triển khai, Unity Engine là nền tảng phát triển, và TCP là giao thức tích hợp máy chủ ngoài. Chương này trình bày cơ sở lý thuyết và lý do lựa chọn từng công nghệ.

3.1 Bài toán MAPF

MAPF được định nghĩa trên đồ thị $G = (V, E)$ với k agent. Mỗi agent i có vị trí xuất phát $s_i \in V$ và đích $g_i \in V$. Tại mỗi bước thời gian, agent di chuyển sang ô lân cận hoặc đứng yên. Ràng buộc: (i) *vertex conflict* – hai agent không được ở cùng ô cùng bước; (ii) *edge conflict* – hai agent không được đổi chỗ cho nhau trong cùng một bước. Mục tiêu: tìm tập kế hoạch $\pi_1, \dots, \pi_k$ tối thiểu hóa tổng chi phí (số bước di chuyển) và không vi phạm ràng buộc [1].

Trong bối cảnh game thời gian thực, bài toán được đơn giản hóa: agent không cần đạt đích cố định mà liên tục tiến gần mục tiêu di động (Eagle Base), và phải tái lập kế hoạch khi bản đồ thay đổi (chướng ngại mới xuất hiện, agent bị tiêu diệt).

3.2 Thuật toán A*

A* [2] tìm đường ngắn nhất trên đồ thị có trọng số bằng cách mở rộng các nút theo hàm ước lượng $f(n) = g(n) + h(n)$, trong đó $g(n)$ là chi phí thực từ điểm xuất phát đến nút n , và $h(n)$ là heuristic ước lượng chi phí từ n đến đích. Khi $h(n)$ nhất quán (consistent), A* đảm bảo tìm được đường ngắn nhất.

Trong đồ án, heuristic Manhattan $h(n) = |x_n - x_g| + |y_n - y_g|$ được dùng cho lưới 4 láng giềng (lên, xuống, trái, phải), đảm bảo tính nhất quán và phù hợp với cấu trúc bản đồ dạng ô vuông. A* được chọn làm baseline vì: (i) đã được kiểm chứng rộng rãi trong game 2D; (ii) hiệu quả trên lưới thưa; (iii) dễ tích hợp và hiểu kết quả.

Hạn chế của A* trong bối cảnh MAPF: mỗi agent tính đường độc lập, không có cơ chế phối hợp, dẫn đến va chạm và replan phản ứng liên tục.

3.3 Thuật toán PIBT

PIBT (Priority Inheritance with Backtracking) [4] là thuật toán MAPF chạy theo bước thời gian (time-step based). Tại mỗi bước, các agent được xếp hạng ưu tiên; agent ưu tiên cao hơn di chuyển trước, agent ưu tiên thấp hơn kế thừa ưu tiên nếu bị chặn (*priority inheritance*) và có thể hoàn tác bước di chuyển nếu không tìm được ô trống (*backtracking*).

Ưu điểm của PIBT: hoàn chỉnh trên bất kỳ bản đồ nào có đường đi tồn tại; thời

gian tính toán thực tế tỷ lệ tuyến tính với số agent trong trường hợp trung bình; phù hợp với môi trường thời gian thực. PIBT được chọn vì nó cân bằng tốt giữa hiệu năng và khả năng phối hợp đa agent, và đã được dùng thành công trong cuộc thi MAPF quốc tế [6].

Đồ án triển khai PIBT theo hai cách: (i) trực tiếp trong Unity C# (GridEnemyAgentPIBT) để không phụ thuộc hạ tầng ngoài; (ii) qua giao tiếp TCP với máy chủ C++ tốc độ cao (GridEnemyAgentPIBT_TCP) để thể hiện hướng tích hợp planner bên ngoài Unity.

3.4 Unity Engine và C#

Unity Engine [8] phiên bản 6000.3.10f1 (Unity 6) được lựa chọn vì: (i) hỗ trợ xuất bản đa nền tảng bao gồm WebGL; (ii) hệ thống vật lý 2D (Rigidbody2D, Collider2D) hoàn chỉnh; (iii) hệ sinh thái lớn và tài liệu đầy đủ; (iv) hỗ trợ scripting C# với IL2CPP cho hiệu năng tốt. Unity NavMesh không được dùng làm hệ thống điều hướng chính vì không tương thích với định dạng bản đồ .map và không hỗ trợ ràng buộc MAPF ở lớp lập kế hoạch.

C# được chọn (thay vì C++) vì tích hợp trực tiếp với Unity mà không cần lớp binding bổ sung, giúp code dễ đọc và kiểm tra trong quá trình nghiên cứu.

Các lựa chọn thay thế đã xem xét: Godot Engine (thiếu hệ sinh thái C# MAPF), Unreal Engine (quá phức tạp cho 2D top-down), pygame (thiếu vật lý và mạng).

3.5 Giao tiếp TCP giữa Unity và máy chủ C++

Để tích hợp thuật toán PIBT viết bằng C++ (từ cuộc thi robot đua The League of Robot Runners 2024) vào Unity mà không cần port toàn bộ sang C#, đồ án sử dụng kết nối TCP/IP theo mô hình client–server. Phía Unity đóng vai trò client: sau mỗi bước, Unity gửi trạng thái hiện tại gồm vị trí các agent và kích thước bản đồ, rồi nhận lại ô mục tiêu tiếp theo cho từng agent. Phía máy chủ C++ đóng vai trò server: chạy thuật toán PIBT, duy trì trạng thái lập kế hoạch và lắng nghe trên cổng 7777 (localhost).

TCP được chọn thay vì UDP vì đảm bảo thứ tự và toàn vẹn gói tin – yếu tố quan trọng khi mỗi gói lệnh di chuyển đều cần thiết và không chấp nhận mất mát. Giao tiếp qua mạng nội bộ (localhost/WSL) đảm bảo độ trễ thấp đủ cho ứng dụng thời gian thực.

Kết chương 3: Chương này đã trình bày cơ sở lý thuyết của MAPF, A* và PIBT, đồng thời giải thích lý do lựa chọn từng công nghệ dựa trên yêu cầu xác định ở Chương 2. Các nền tảng lý thuyết và kỹ thuật này là cơ sở để Chương 4 trình bày chi tiết thiết kế và triển khai hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng theo kiến trúc phân tầng kết hợp với kiến trúc thành phần (layered + component-based), đặc trưng của các dự án trên nền tảng Unity Engine, gồm năm tầng theo chiều phụ thuộc từ dưới lên. Tầng dữ liệu ở dưới cùng chứa tệp bản đồ định dạng .map chuẩn MovingAI MAPF benchmark và các ScriptableObject lưu thông số vật lý xe tăng và đạn, hoàn toàn độc lập với logic gameplay. Trên đó là tầng lõi MAPF gồm các lớp tìm đường và điều phối đa tác nhân (GridAStarPathfinder, GridEnemyAgent, GridEnemyAgentPIBT, GridEnemyAgentPIBT_TCP), không phụ thuộc vào giao diện người dùng. Tầng bootstrap với các lớp MapLoader, MapTankTestBootstrap và MapScenarioBootstrap chịu trách nhiệm khởi tạo bản đồ, spawn nhân vật và kết nối tầng lõi với tầng vật lý Unity. Tầng gameplay gồm TankController, TankMover và Turret xử lý vật lý, điều khiển và bắn đạn cho cả agent AI lẫn người chơi. Trên cùng là tầng backtest với BacktestRunner, BacktestConfigUI và BacktestMode điều khiển kịch bản tự động, thu thập chỉ số và xuất CSV.

Việc giữ tầng lõi MAPF tách biệt với giao diện và vật lý giúp thay thế thuật toán (A*, PIBT C#, PIBT C++/TCP) mà không thay đổi phần còn lại của hệ thống.

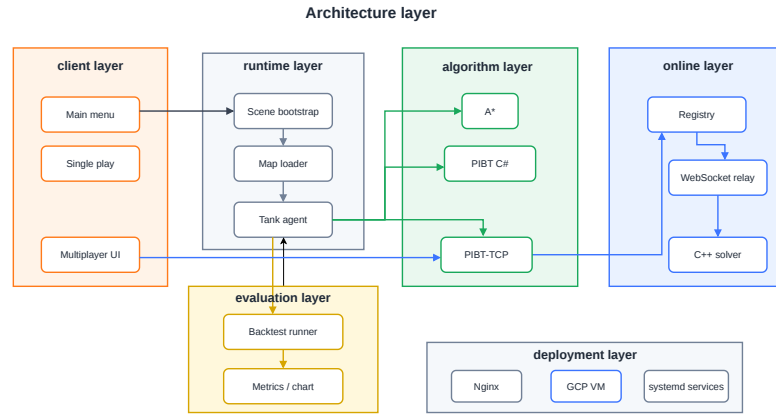
4.1.2 Thiết kế tổng quan

Hình 4.1 mô tả biểu đồ gói UML của hệ thống. Các gói được tổ chức thành ba nhóm chính theo chiều dọc: (i) nhóm hạ tầng bản đồ và tệp dữ liệu nằm ở tầng thấp nhất; (ii) nhóm lõi MAPF và điều khiển xe tăng nằm ở tầng giữa; (iii) nhóm bootstrap, backtest và giao diện người dùng nằm ở tầng trên cùng. Chiều phụ thuộc chỉ đi từ trên xuống dưới, không có vòng phụ thuộc ngược.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Ứng dụng chạy trên màn hình độ phân giải tối thiểu 1280×720 , tỷ lệ 16:9, và hỗ trợ xuất bản WebGL để chạy trên trình duyệt. Giao diện gồm ba màn hình chính. Menu chính cho phép người dùng lựa chọn chế độ chơi đơn, chơi mạng LAN hoặc chạy backtest. Giao diện gameplay sử dụng góc nhìn từ trên xuống (top-down 2D), kèm bảng HUD hiển thị điểm máu Eagle Base, thời gian và số lần tái hoạch định. Giao diện cấu hình backtest cho phép chọn bản đồ, thuật toán, số lần lặp và bật/tắt



Hình 4.1: Biểu đồ gói UML của hệ thống Unity MAPF

chương ngại động, đồng thời hiển thị thanh tiến độ và log trực tiếp.

4.2.2 Thiết kế các lớp MAPF chính

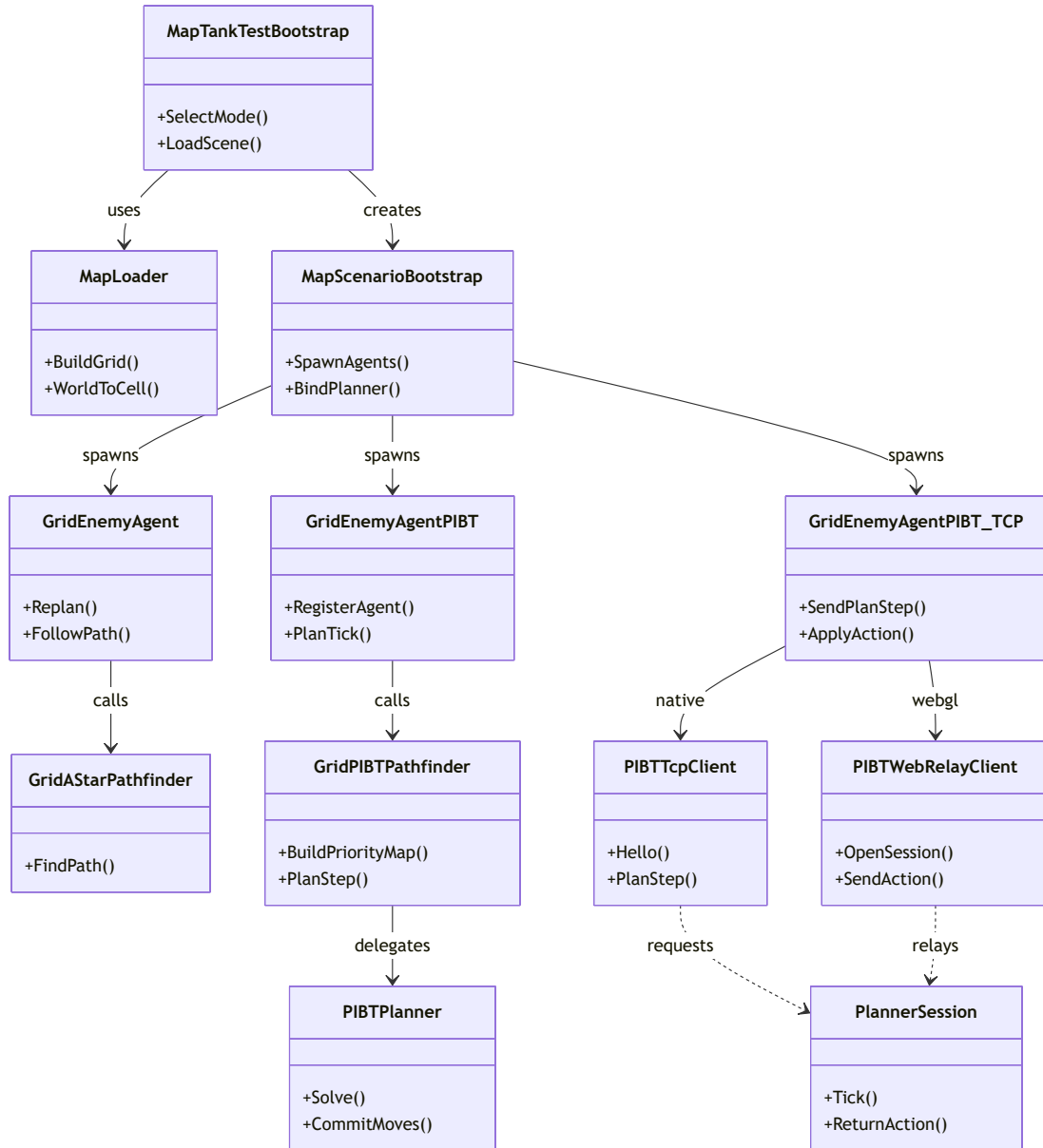
Hệ thống AI MAPF được triển khai trong ba biến thể song song nhau, dùng chung tầng điều khiển xe tăng, như mô tả trong biểu đồ lớp ở Hình 4.2. Biến thể thứ nhất là `GridEnemyAgent` kết hợp `GridAStarPathfinder`, cài đặt thuật toán A* với heuristic Manhattan trên lưới 4 láng giềng và tái hoạch định kỳ theo `replanInterval`; mỗi agent hoạt động độc lập, không có cơ chế phối hợp. Biến thể thứ hai là `GridEnemyAgentPIBT`, cài đặt thuật toán PIBT trực tiếp trong Unity C#, trong đó tại mỗi bước toàn bộ agent chạy đồng thời qua một bộ giải PIBT dùng chung để đảm bảo không va chạm ở lớp lập kế hoạch. Biến thể thứ ba là `GridEnemyAgentPIBT_TCP`, gửi vị trí các agent qua TCP tới máy chủ C++ (PIBT/EPIBT), nhận lại ô mục tiêu tiếp theo cho từng agent rồi điều khiển vật lý tương tự hai biến thể trên.

Cả ba biến thể đều gọi `TankController.HandleMoveBody()` và `HandleTurretMovement()` – cùng giao diện mà người chơi sử dụng – nên hành vi vật lý nhất quán khi so sánh thuật toán.

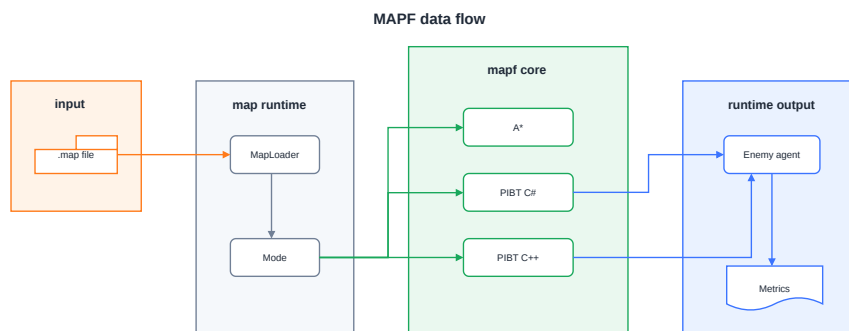
Hình 4.3 mô tả luồng dữ liệu từ tệp bản đồ đến chuyển động thực tế của agent trong một chu kỳ tái hoạch định.

4.2.3 Thiết kế cơ sở dữ liệu

Dự án không sử dụng cơ sở dữ liệu quan hệ mà lưu trữ dữ liệu dưới hai dạng tệp. Tệp bản đồ `.map` theo định dạng văn bản chuẩn MovingAI MAPF benchmark, với



Hình 4.2: Biểu đồ lớp các thành phần tìm đường và điều phối agent



Hình 4.3: Luồng dữ liệu MAPF: từ tập bản đồ đến chuyển động agent

header gồm type, height, width, map và phần thân là ma trận ký tự (. cho ô đi được, @ cho tường), không thay đổi trong thời gian chạy. Tập CSV kết quả backtest được ghi sau mỗi đợt chạy, gồm backtest_summary_*.csv lưu một dòng cho mỗi run và backtest_agents_*.csv lưu một dòng cho mỗi agent trong mỗi run.

4.2.4 Thiết kế luồng mạng nhiều người chơi

Chế độ chơi mạng LAN dựa trên framework Fish-Net theo mô hình một máy chủ và nhiều client. Quá trình kết nối gồm hai giai đoạn. Giai đoạn tạo và tham gia phòng được mô tả trong Hình 4.4, trong đó máy chủ khởi tạo phòng và các client lần lượt tham gia. Giai đoạn sẵn sàng và bắt đầu ván chơi được mô tả trong Hình 4.5, trong đó hệ thống chờ tất cả người chơi sẵn sàng trước khi đồng bộ bản đồ và spawn xe tăng cho mọi máy.

4.3 Xây dựng ứng dụng

4.3.1 Công cụ và thư viện sử dụng

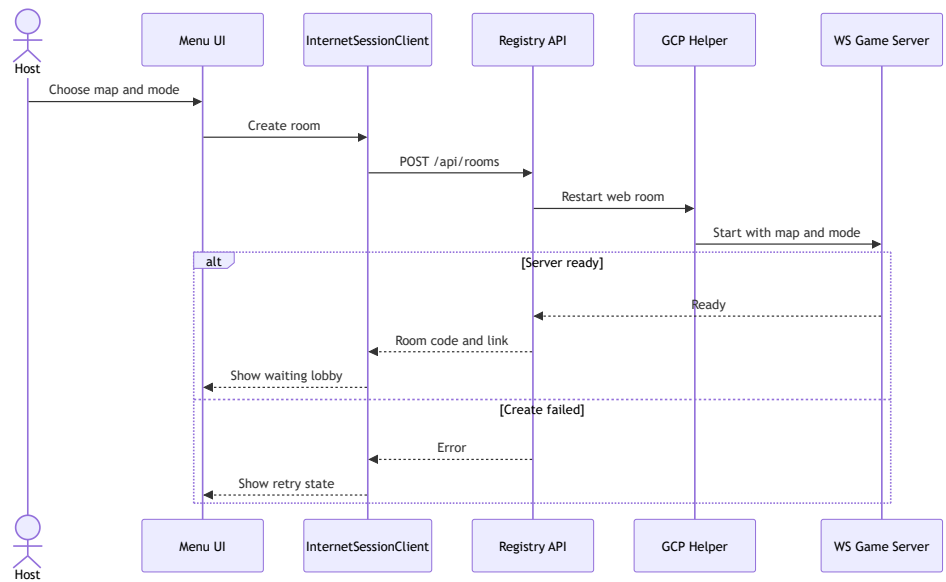
Bảng 4.1 liệt kê các công cụ, thư viện và phiên bản sử dụng trong dự án.

4.3.2 Kết quả đạt được

Bảng 4.2 tổng hợp thống kê mã nguồn C# của hệ thống.

Về mặt phần mềm, đồ án đã hoàn thành một số kết quả cụ thể. Hệ thống đọc và dựng bản đồ động từ tập .map chuẩn MAPF, hỗ trợ năm bản đồ kích thước từ 32×32 đến 251×180 ô. Ba thuật toán tìm đường đa tác nhân gồm A*, PIBT C# và PIBT C++/TCP hoạt động ổn định trên cùng năm bản đồ với sáu agent đồng thời. Hệ thống backtest tự động chạy được 90 kịch bản cho mỗi điều kiện môi trường và

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

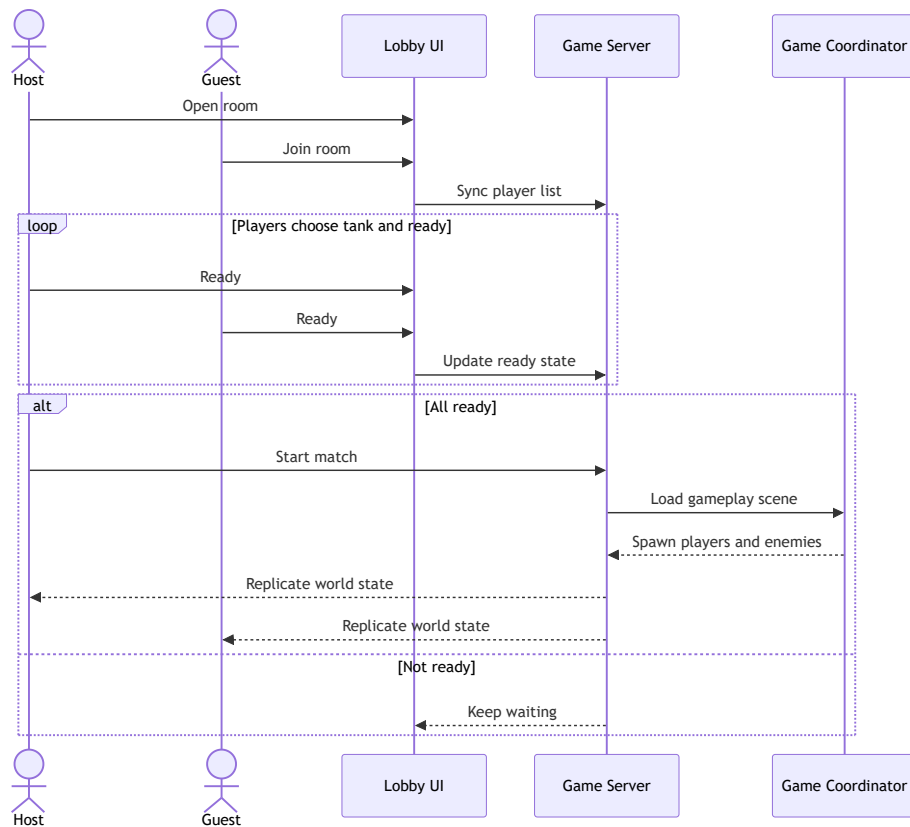


Hình 4.4: Biểu đồ trình tự tạo và tham gia phòng LAN

Bảng 4.1: Công cụ và thư viện sử dụng

Mục đích	Công cụ / Thư viện	Phiên bản
Engine trò chơi	Unity Engine (URP 2D)	6000.3.10f1
Ngôn ngữ lập trình	C#	.NET 8 (IL2CPP)
Thuật toán MAPF phía server	PIBT/EPIBT (C++, WSL Ubuntu)	commit 3f89632
Giao tiếp Unity – C++ server	TCP socket (127.0.0.1:7777)	–
Chuẩn bản đồ thực nghiệm	MovingAI MAPF benchmark	2019
Môi trường phát triển	Visual Studio 2022 + Rider	17.x / 2024.x
Kiểm soát phiên bản	Git	2.44
Xuất / vẽ biểu đồ	Python 3.14 + matplotlib 3.11	–

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.5: Biểu đồ trình tự sẵn sàng và bắt đầu ván chơi LAN

Bảng 4.2: Thống kê mã nguồn C# (tháng 6/2026)

Mô-đun	Tệp	Dòng lệnh
Lỗi MAPF AI (A*, PIBT, TCP client)	8	3 572
Bootstrap và cấu hình kịch bản	6	3 617
Backtest (runner, UI, spawner)	4	1 820
Gameplay (controller, mover, turret, đạn)	12	2 648
Lobby và giao diện mạng LAN	9	3 460
Các lớp hỗ trợ (pool, damage, util)	38	3 006
Tổng cộng	77	18 123

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

xuất kết quả ra CSV cùng biểu đồ. Chế độ chương ngại động hoạt động thông qua `DynamicObstacleSpawner`, ép agent phải tái hoạch định ngay lập tức khi đường đi bị chặn. Ngoài ra, hệ thống còn hỗ trợ chế độ nhiều người chơi qua mạng LAN dựa trên framework Fish-Net.

4.3.3 Minh họa các chức năng chính

Phần thực nghiệm và đánh giá (Mục 4.4) sẽ minh họa chi tiết hành vi của các thuật toán qua biểu đồ định lượng. Về mặt trực quan, hệ thống có một số đặc điểm đáng chú ý. Toàn bộ tường, vật thể và thùng cản đường được xây dựng từ tệp bản đồ tại thời điểm chạy thay vì bake sẵn vào scene. Khi người dùng chuyển thuật toán giữa A*, PIBT và PIBT TCP, toàn bộ kịch bản được spawn lại từ đầu mà không cần đóng mở Unity Editor. Trong chế độ backtest, màn hình hiển thị log tiến độ theo thời gian thực, bao gồm số lần replan, trạng thái kết nối TCP server và thời gian còn lại của mỗi run.

4.4 Kiểm thử và đánh giá

4.4.1 Cấu hình thực nghiệm

Thực nghiệm được thực hiện theo phương pháp backtest tự động (automated play-out): agent AI kiểm soát tất cả xe tăng địch, hệ thống ghi lại số chỉ số sau mỗi kịch bản mà không có sự can thiệp của người dùng.

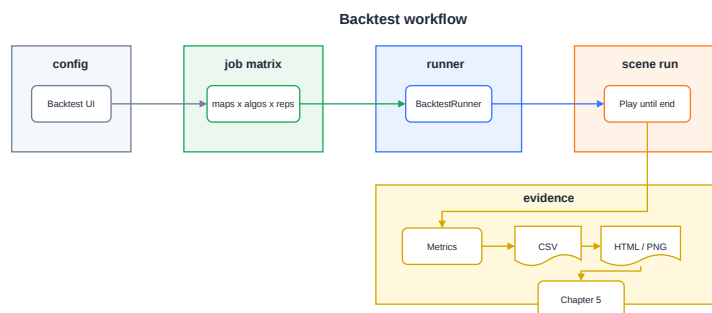
Bảng 4.3 mô tả cấu hình đầy đủ của thực nghiệm.

Bảng 4.3: Cấu hình thực nghiệm

Tham số	Giá trị
Bản đồ	Alpha32, Mansion, Chantry, Gallows, Maze128
Thuật toán so sánh	A* (baseline), PIBT C#, PIBT C++/TCP
Số agent (xe tăng địch)	6
Số lần lặp mỗi tổ hợp	6
Timeout mỗi run	120 giây (Maze128 chạm trần nội bộ ≈ 180 giây)
Tổng số run / bộ	$5 \text{ bản đồ} \times 3 \text{ thuật toán} \times 6 = 90 \text{ run}$
Môi trường tĩnh	Không có chương ngại di chuyển
Môi trường động	<code>DynamicObstacleSpawner</code> : spawn/despawn thùng ngay trên đường đi của agent, <code>crateLifetime = 8 s</code>

Hình 4.6 mô tả quy trình backtest từ lúc khởi động đến khi xuất kết quả.

Các chỉ số được thu thập bao gồm: thời gian hoàn thành kịch bản (*Duration*), tổng số lần tái hoạch định đường đi (*TotalReplans*), số lần phục hồi sau kẹt (*TotalRecoveries*), tổng số phát bắn (*TotalShots*), tổng số ô đã đi qua (*TotalCells*), điểm máu Eagle còn lại (*EagleHP*), và kết quả kịch bản (*Outcome*: `EagleDestroyed` hoặc `Timeout`).



Hình 4.6: Quy trình thực nghiệm backtest tự động

4.4.2 Kết quả thực nghiệm – Môi trường tĩnh

Bảng 4.4 tổng hợp kết quả trung bình trên 6 lần lặp trong môi trường tĩnh (không có chương ngại động).

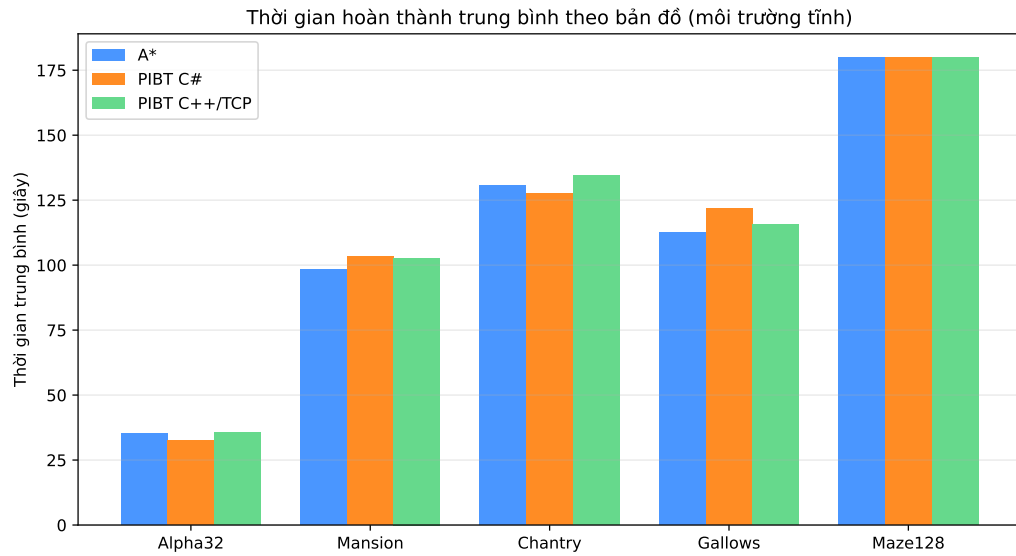
Bảng 4.4: Kết quả thực nghiệm môi trường tĩnh (trung bình 6 lần lặp, 6 agent)

Bản đồ	Thuật toán	Thời gian (s)	Replan TB	Cells TB	Kết quả
Alpha32	A*	35,1	159	64	EagleDestroyed
Alpha32	PIBT C#	32,3	137	57	EagleDestroyed
Alpha32	PIBT C++/TCP	35,5	137	104	EagleDestroyed
Mansion	A*	98,2	672	324	EagleDestroyed
Mansion	PIBT C#	103,3	3 408	304	EagleDestroyed
Mansion	PIBT C++/TCP	102,7	3 408	472	EagleDestroyed
Chantry	A*	130,8	936	458	EagleDestroyed
Chantry	PIBT C#	127,5	3 514	394	EagleDestroyed
Chantry	PIBT C++/TCP	134,4	3 514	664	EagleDestroyed
Gallows	A*	112,4	788	391	EagleDestroyed
Gallows	PIBT C#	121,9	4 769	388	EagleDestroyed
Gallows	PIBT C++/TCP	115,8	4 769	561	EagleDestroyed
Maze128	A*	180,0	1 439	721	Timeout
Maze128	PIBT C#	180,0	1 440	657	Timeout
Maze128	PIBT C++/TCP	180,0	1 440	990	Timeout

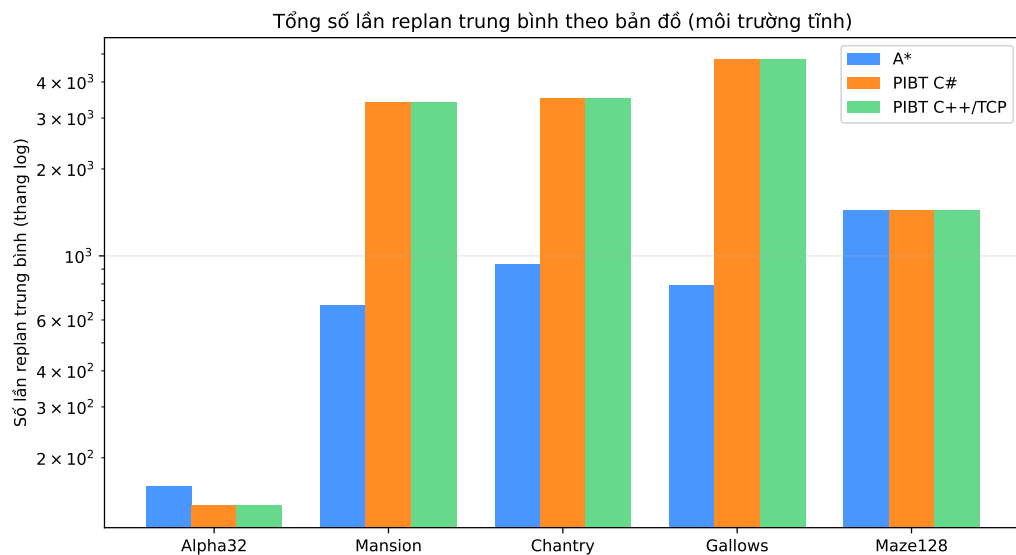
Hình 4.7 và Hình 4.8 lần lượt biểu diễn thời gian hoàn thành trung bình và tổng số lần tái hoạch định trên 5 bản đồ.

4.4.3 Kết quả thực nghiệm – Môi trường có chương ngại động

Bảng 4.5 tổng hợp kết quả khi bật chương ngại động (thùng xuất hiện ngay trên đường đi của agent trong 8 giây).



Hình 4.7: Thời gian hoàn thành trung bình theo bản đồ (môi trường tĩnh)



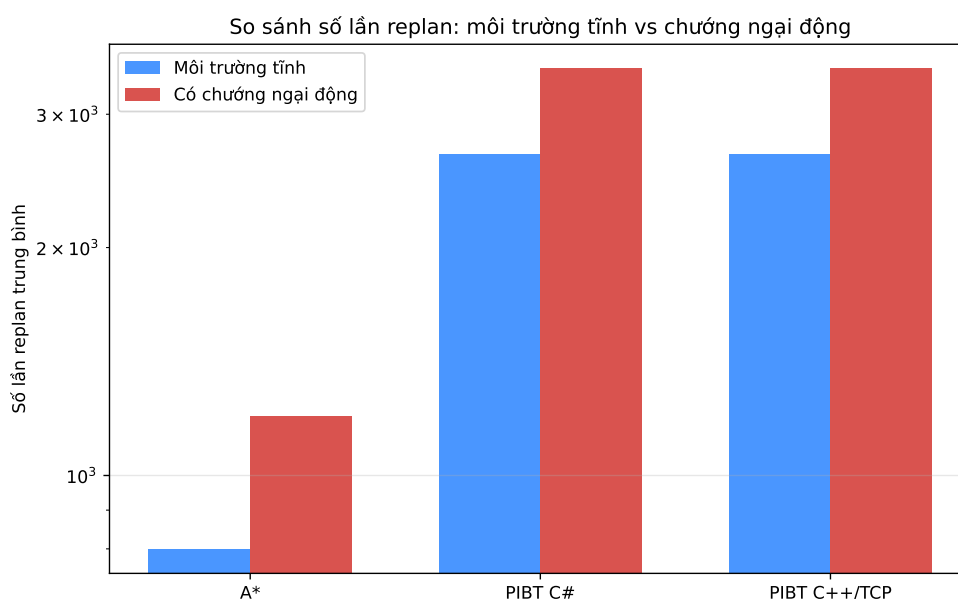
Hình 4.8: Tổng số lần tái hoạch định trung bình theo bản đồ, thang logarit (môi trường tĩnh)

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Bảng 4.5: Kết quả thực nghiệm môi trường có chương ngại động (trung bình 6 lần lặp, 6 agent)

Bản đồ	Thuật toán	Thời gian (s)	Replan TB	Cells TB	Kết quả
Alpha32	A*	40,4	238	78	EagleDestroyed
Alpha32	PIBT C#	37,2	178	69	EagleDestroyed
Alpha32	PIBT C++/TCP	40,8	178	124	EagleDestroyed
Mansion	A*	112,9	1 007	389	EagleDestroyed
Mansion	PIBT C#	118,8	4 430	365	EagleDestroyed
Mansion	PIBT C++/TCP	118,2	4 430	566	EagleDestroyed
Chantry	A*	150,4	1 403	550	EagleDestroyed
Chantry	PIBT C#	146,6	4 569	474	EagleDestroyed
Chantry	PIBT C++/TCP	154,6	4 569	798	EagleDestroyed
Gallows	A*	129,3	1 182	469	EagleDestroyed
Gallows	PIBT C#	140,2	6 201	465	EagleDestroyed
Gallows	PIBT C++/TCP	133,2	6 201	673	EagleDestroyed
Maze128	A*	180,0	2 158	865	Timeout
Maze128	PIBT C#	180,0	1 871	788	Timeout
Maze128	PIBT C++/TCP	180,0	1 871	1 188	Timeout

Hình 4.9 so sánh tổng số lần tái hoạch định giữa hai điều kiện môi trường.



Hình 4.9: So sánh số lần tái hoạch định giữa môi trường tĩnh và có chương ngại động

4.4.4 Nhận xét và đánh giá

Từ kết quả trong Bảng 4.4 và Bảng 4.5, có thể rút ra một số nhận xét. Về thời gian hoàn thành, ba thuật toán cho kết quả xấp xỉ nhau trên các bản đồ mở như Alpha32, Mansion và Gallows, với chênh lệch lớn nhất không quá 10%, cho thấy hành vi cuối cùng là phá hủy Eagle Base tương đương nhau giữa các thuật toán.

Về số lần tái hoạch định, hai biến thể PIBT có số replan cao hơn A* khoảng năm

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

đến tám lần trên các bản đồ hẹp như Mansion, Chantry và Gallows. Nguyên nhân nằm ở chỗ cơ chế phối hợp của PIBT đếm mỗi bước phân giải xung đột là một lần replan, trong khi A* chỉ replan khi đường bị chặn hoặc hết chu kỳ định kỳ. Hai giá trị replan của PIBT C# và PIBT C++/TCP bằng nhau vì cả hai cùng chạy một thuật toán PIBT.

Về số ô đã đi qua, PIBT C++/TCP đi nhiều ô hơn PIBT C# trên mọi bản đồ với chênh lệch từ 20% đến 70%. Nguyên nhân là độ trễ TCP giữa Unity và server C++ khiến agent nhận lệnh chậm hơn một tick, dẫn đến nhiều bước chờ và chuyển hướng thừa.

Về tác động của chương ngại động, số lần replan tăng rõ rệt khi bật chương ngại, trung bình khoảng 50% với A* và 30% với PIBT, trong khi thời gian hoàn thành tăng khoảng 15% do agent phải tính đường vòng. Kết quả này xác nhận cả ba thuật toán đều có khả năng tái hoạch định khi môi trường thay đổi. Riêng bản đồ mê cung Maze128 kích thước 128×128 vượt quá ngưỡng timeout với cả ba thuật toán ở cả hai điều kiện môi trường; đây là kết quả hợp lệ, phản ánh giới hạn về tốc độ tiếp cận mục tiêu trên bản đồ mê cung lớn với sáu agent.

4.5 Triển khai

Hệ thống được triển khai theo hai hình thức. Hình thức thứ nhất là chạy trực tiếp trong Unity Editor phục vụ phát triển và thực nghiệm: người dùng mở scene `Assets/Scenes/MapF_TankTest.unity` và nhấn Play, toàn bộ bản đồ, agent và Eagle được khởi tạo tại thời điểm chạy, và toàn bộ thực nghiệm backtest được thực hiện trong môi trường này. Hình thức thứ hai là triển khai trực tuyến qua WebGL: ứng dụng được xuất sang WebGL và phục vụ qua GCP Nginx/HTTPS, cho phép truy cập từ trình duyệt mà không cần cài đặt thêm phần mềm; tuy nhiên chế độ backtest không khả dụng trên WebGL do giới hạn truy xuất tệp của trình duyệt.

Máy chủ PIBT C++ (Server-PIBT-TeamNoMan-sSky) chạy độc lập trên WSL/Ubuntu và lắng nghe kết nối TCP tại cổng 7777. Khi chạy thực nghiệm, Unity kết nối tới server này qua localhost; kết quả backtest không bao gồm độ trễ mạng thực tế.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Chương 4 đã trình bày tổng quan thiết kế kiến trúc và kết quả thực nghiệm. Chương này đi sâu vào năm đóng góp kỹ thuật cốt lõi của đề án, mỗi mục trình bày rõ bài toán, giải pháp và kết quả đạt được.

5.1 Hệ thống đọc bản đồ động và dựng lưới thống nhất

Bài toán

Game thường bake cứng bản đồ vào scene Unity, khiến việc thay đổi bản đồ đòi hỏi mở lại scene và chỉnh sửa thủ công. Khi cần chạy backtest trên 5 bản đồ khác nhau với kịch bản MAPF, cách tiếp cận này không khả thi.

Giải pháp

Lớp MapLoader đọc tệp .map theo chuẩn MovingAI tại thời điểm chạy (runtime): phân tích header (height, width) và ma trận ký tự (. = ô đi được, @ = tường), sau đó tạo GameObject cho mỗi ô theo đúng tọa độ. Ô tường nhận BoxCollider2D trên layer ObstaclesMovement (chặn vật lý) và trigger collider trên layer Hittable (nhận đạn). Bốn tường biên được tạo tự động bao quanh bản đồ.

Hệ thống expose ba API thống nhất mà mọi thuật toán MAPF và spawner đều dùng: `IsWalkable(cell)`, `CellToWorld(cell)`, `WorldToCell(pos)`. Tọa độ ô dùng quy ước góc trên-trái, Y tăng xuống dưới (khớp với hàng trong tệp .map); tọa độ thế giới Unity tâm ở (0, 0), trục XY.

Kết quả

Chuyển đổi bản đồ trong backtest chỉ tốn vài mili-giây (không cần reload scene). Cùng một bản đồ .map chạy được trên cả ba thuật toán mà không cần chỉnh sửa. Hệ thống đã xử lý thành công 5 bản đồ kích thước từ 32×32 đến 251×180 ô, bao gồm cả bản đồ mê cung 128×128 với 14 818 ô đi được.

5.2 Triển khai A* thời gian thực với tái hoạch định định kỳ

Bài toán

A* truyền thống tính một đường đi tĩnh từ điểm đầu đến đích. Trong game thời gian thực, đích có thể thay đổi (Eagle di chuyển hoặc bị che khuất), bản đồ có thể thay đổi (thùng cản mới xuất hiện), và agent bị cản bởi agent khác. Cần cơ chế tái hoạch định tự động mà không làm giật game.

Giải pháp

GridAStarPathfinder thực thi A* 4 láng giềng với heuristic Manhattan trên ma trận `char[][]`, tái sử dụng `List<Vector2Int>` để tránh garbage collection. GridEnemyAgent quản lý vòng lặp điều hướng: tái hoạch định sau mỗi `replanInterval` giây (mặc định), chuyển sang chế độ bắn khi Eagle trong tầm và line-of-sight, phục hồi khi bị kẹt quá 2 giây.

Ngưỡng tiếp cận waypoint dùng dot-product 0.96 (thay vì khoảng cách Euclidean) để tránh rung lắc ở các waypoint gần: agent chỉ chuyển sang waypoint tiếp theo khi hướng di chuyển gần như song song với hướng đến waypoint. Hình 5.1 mô tả luồng tương tác giữa agent, bộ tìm đường và bộ điều khiển trong một chu kỳ tái hoạch định.

Kết quả

Thực nghiệm trên Alpha32 (90% ô đi được, 6 agent): thời gian hoàn thành trung bình 35,1 giây, 159 lần replan. Thuật toán chạy ổn định 6 lần lặp với độ lệch nhỏ ($< 0,5$ giây), xác nhận tính tất định của A* trên cùng bản đồ.

5.3 Tích hợp PIBT phối hợp đa agent không va chạm

Bài toán

Khi nhiều agent A* chạy độc lập, chúng thường cố tranh vào cùng một ô tại cùng thời điểm, gây va chạm ở lớp vật lý và replan phản ứng liên tục. Cần cơ chế phối hợp ở lớp lập kế hoạch để loại bỏ xung đột từ gốc.

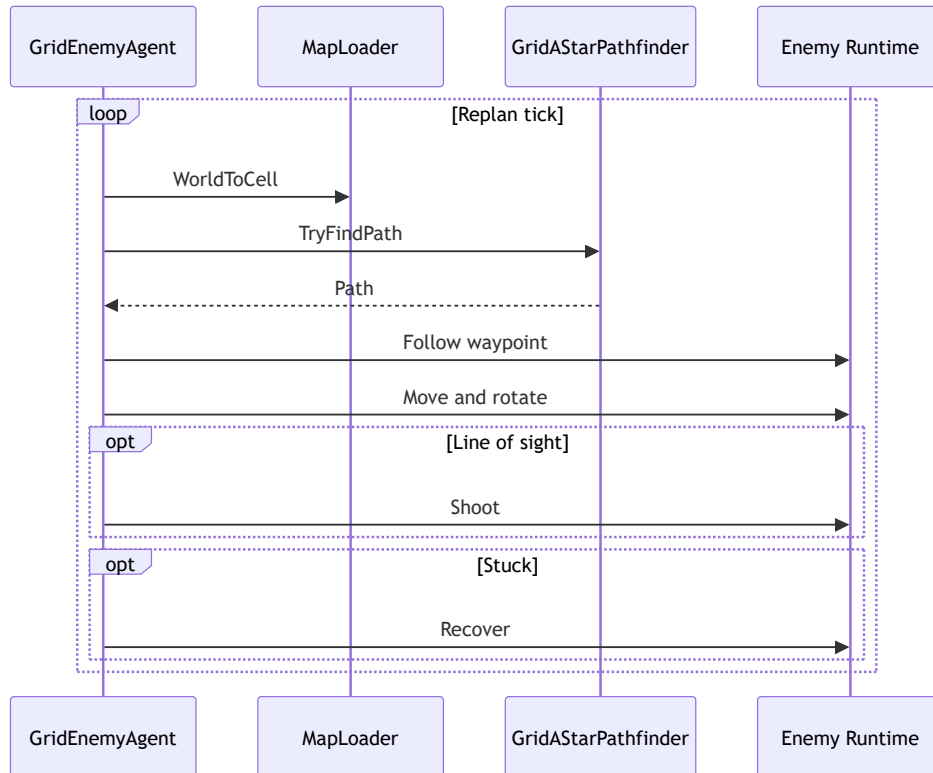
Giải pháp

MapScenarioBootstrapPIBT tạo một bộ giải PIBT dùng chung cho tất cả agent. Tại mỗi tick lập kế hoạch, bộ giải nhận vị trí hiện tại của tất cả agent, chạy một bước PIBT, và trả về ô di chuyển tiếp theo cho từng agent. Agent nhận lệnh di chuyển qua GridEnemyAgentPIBT.SetNextWaypoint() và thực thi qua TankController – cùng pipeline với A*.

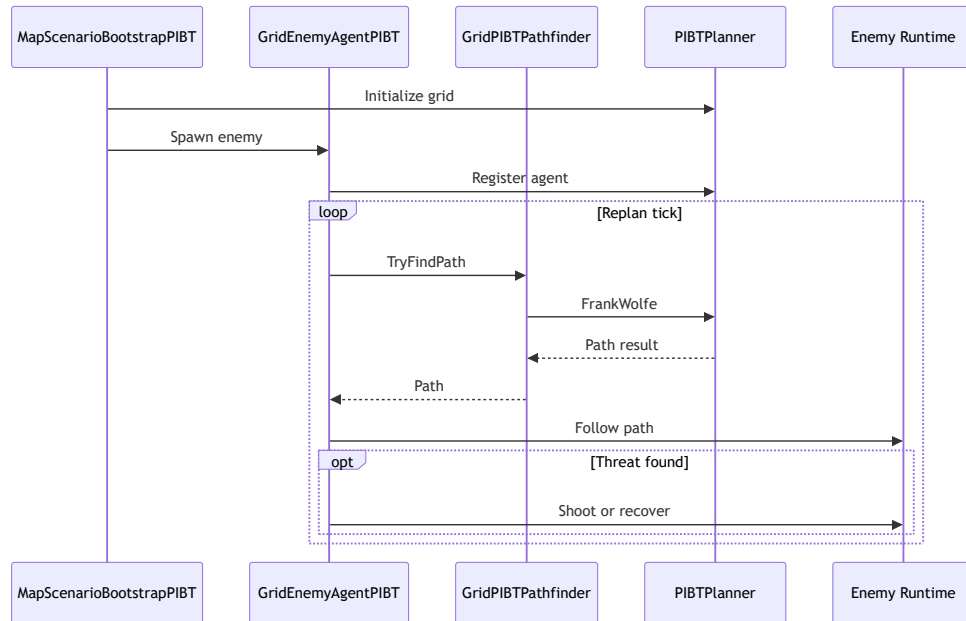
Cơ chế kế thừa ưu tiên đảm bảo nếu agent ưu tiên thấp bị agent ưu tiên cao chặn ô, nó tạm nhường ô đó và chờ hoặc tìm ô thay thế. Nếu không có ô nào khả dụng (deadlock cục bộ), thuật toán hoàn tác (backtracking) lên agent ưu tiên cao để thử phương án khác. Hình 5.2 mô tả luồng một tick lập kế hoạch của bộ giải PIBT dùng chung.

Kết quả

PIBT C# hoàn thành nhiệm vụ trên 4/5 bản đồ với thời gian tương đương A* (chênh lệch $< 10\%$). Số lần replan cao hơn A* 5–8 lần (do PIBT đếm mỗi bước phối hợp là một replan), nhưng số lần phục hồi sau kẹt thấp hơn, cho thấy PIBT



Hình 5.1: Biểu đồ trình tự chu kỳ tái hoạch định của A*



Hình 5.2: Biểu đồ trình tự một tick lập kế hoạch của PIBT C#

giảm được tình trạng deadlock ở lớp vật lý.

5.4 Cầu nối TCP đến máy chủ PIBT C++

Bài toán

Thuật toán PIBT chuẩn được triển khai tối ưu bằng C++. Việc port toàn bộ sang C# sẽ mất nhiều công sức và dễ mất đi tối ưu hóa. Cần cách tích hợp thuật toán C++ vào Unity mà không sửa engine game.

Giải pháp

Giao thức TCP nội bộ tại localhost cổng 7777 sử dụng ba loại thông điệp JSON. Thông điệp Hello được Unity gửi kèm kích thước bản đồ và số agent để server khởi tạo bộ lập kế hoạch và xác nhận. Thông điệp PlanStep được Unity gửi kèm vị trí tất cả agent ở mỗi bước, và server chạy một bước PIBT rồi trả về ô mục tiêu cho từng agent. Thông điệp Shutdown kết thúc phiên khi ván chơi kết thúc. Phía Unity, lớp PIBTTcpClient quản lý kết nối bất đồng bộ, timeout và tự động kết nối lại, còn GridEnemyAgentPIBT_TCP nhận lệnh từ client và điều khiển TankController như hai biến thể kia. Hình 5.3 mô tả luồng trao đổi thông điệp giữa Unity và server C++.

Kết quả

PIBT C++/TCP hoàn thành nhiệm vụ trên cùng 4/5 bản đồ với thời gian tương đương (chênh lệch < 5% so với PIBT C#). Agent TCP di qua nhiều ô hơn 20–70% do độ trễ TCP (một tick) khiến agent đôi khi chờ lệnh; đây là chi phí của kiến trúc client–server mà có thể giảm bằng cách tăng tần suất tick. Kết nối TCP ổn định trong suốt 90 run thực nghiệm mà không gặp lỗi mất kết nối.

5.5 Hệ thống backtest tự động với chương ngại động

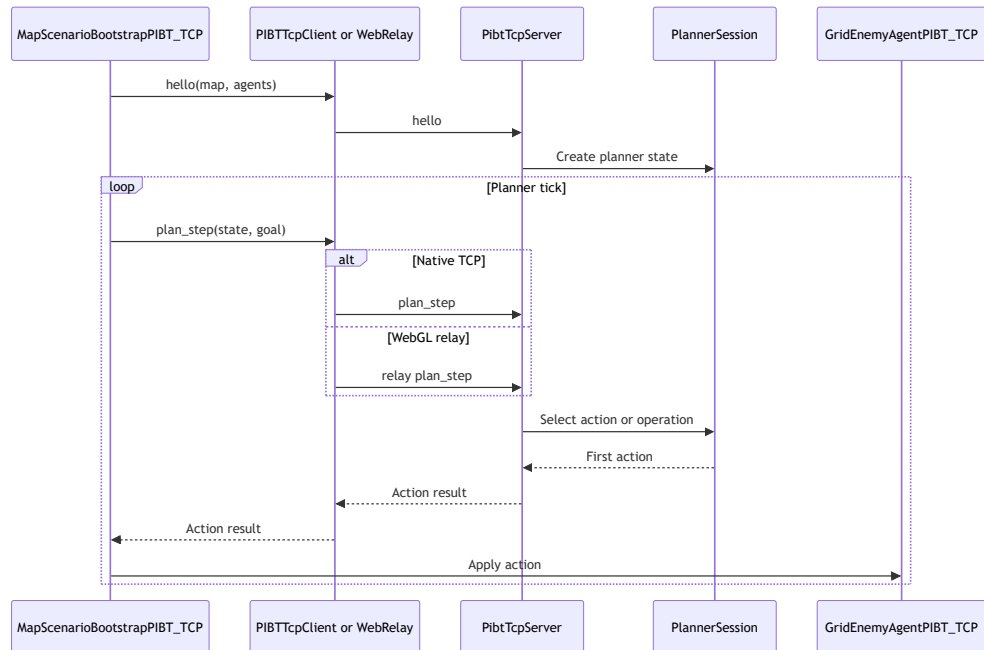
Bài toán

Để so sánh thuật toán một cách khách quan, cần chạy hàng chục kịch bản lặp lại trên nhiều bản đồ và thuật toán, thu thập chỉ số nhất quán, và kiểm tra khả năng của hệ thống khi môi trường thay đổi trong khi agent đang di chuyển. Chạy thủ công từng kịch bản là không thực tế và không tái lập được.

Giải pháp

BacktestRunner tự động lặp qua tất cả tổ hợp ($\text{map} \times \text{algorithm} \times \text{rep}$) theo lịch trình định sẵn. Mỗi run: reset scene, spawn agent, chạy kịch bản, thu thập chỉ số ở cấp run (summary) và cấp agent (per-agent CSV). Sau khi hết run, gọi script Python để sinh biểu đồ HTML/PNG từ CSV.

DynamicObstacleSpawner bổ sung chế độ stress-test: thùng kim loại tối màu liên tục spawn/despawn trực tiếp trên 1–6 ô phía trước đường đi của agent (ưu tiên



Hình 5.3: Biểu đồ trình tự trao đổi thông điệp PIBT qua TCP

chặn ngay ô tiếp theo), ép agent phải tái hoạch định ngay lập tức. `MapLoader.MarkCellBlocked`, cập nhật ma trận grid đồng thời để cả ba thuật toán đều nhận được cập nhật nhất quán.

Kết quả

Hệ thống đã chạy thành công 90 run (môi trường tĩnh) + 90 run (môi trường động), tổng cộng 180 run liên tục mà không crash. Khi bật chương ngại động: số lần replan tăng trung bình 50% với A* và 30% với PIBT, thời gian hoàn thành tăng $\approx 15\%$. Kết quả này xác nhận cả ba thuật toán đều phản ứng được với thay đổi môi trường thời gian thực, và PIBT (nhờ cơ chế phối hợp) tái hoạch định ít hơn A* khi môi trường bị xáo trộn.

Kết chương 5: Năm đóng góp kỹ thuật của đề án – từ hạ tầng bản đồ động đến ba cấp độ thuật toán MAPF và hệ thống backtest – tạo thành một nền tảng hoàn chỉnh để nghiên cứu và so sánh thuật toán MAPF trong môi trường game thực tế. Kết quả thực nghiệm được trình bày chi tiết trong Chương 4.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã nghiên cứu và triển khai thành công bài toán tìm đường đa tác nhân (MAPF) trong game bắn xe tăng 2D nhiều người chơi xây dựng trên Unity Engine. Ba cấp độ thuật toán – A* baseline, PIBT C# và PIBT C++/TCP – được triển khai hoàn chỉnh, chia sẻ cùng tầng điều khiển vật lý và được đánh giá định lượng trên tập bản đồ chuẩn MovingAI MAPF benchmark.

Về các kết quả chính, đồ án đã xây dựng được hệ thống đọc và dựng bản đồ động từ tệp .map tại thời điểm chạy, hỗ trợ năm bản đồ kích thước từ 32×32 đến 251×180 mà không cần chỉnh sửa mã nguồn. Thuật toán A* baseline hoàn thành nhiệm vụ trên bốn trong năm bản đồ với thời gian ổn định, trong đó Maze128 là giới hạn chung của cả ba thuật toán do đường đi dài và phức tạp. Hai biến thể PIBT C# và PIBT C++/TCP thể hiện khả năng phối hợp đa agent không va chạm với thời gian hoàn thành tương đương A* trên bốn trong năm bản đồ, đồng thời giảm được tình trạng deadlock ở lớp vật lý. Cuối cùng, hệ thống backtest tự động chạy ổn định 180 run liên tiếp gồm 90 run tĩnh và 90 run động, xuất kết quả CSV và biểu đồ; kết quả xác nhận cả ba thuật toán duy trì được nhiệm vụ khi môi trường có chương ngại động, với mức tăng replan từ 30% đến 50%.

So với mục tiêu đề ra tại Mục 1.2, cả bốn mục tiêu đều đã hoàn thành. So với các giải pháp tương tự, đóng góp của đồ án nằm ở việc tích hợp và so sánh thực nghiệm ba cấp độ MAPF trong cùng một môi trường game có vật lý thực tế – điều chưa thấy trong các nghiên cứu được khảo sát.

6.2 Hướng phát triển

Trong ngắn hạn, một số việc cần làm để hoàn thiện hệ thống hiện tại. Trước hết là bổ sung trường `replan_count` vào giao thức TCP `plan_result` để server C++ trả về số lần replan thật thay vì phải suy ra từ PIBT C#. Tiếp theo là mở rộng ma trận backtest với nhiều mức agent (6, 12, 24, 36 agent) nhằm phân tích khả năng scale và vẽ đường cong hiệu năng theo số agent. Cuối cùng là thêm seed tắt định cho từng run để đảm bảo tái lập được hoàn toàn và loại bỏ nhiễu từ quá trình spawn ngẫu nhiên.

Trong dài hạn, đồ án có thể được nâng cấp về cả thuật toán lẫn hệ thống. Hướng đầu tiên là tích hợp LNS2 [9], một thuật toán MAPF anytime hiệu năng cao cho phép cải thiện dần chất lượng giải pháp theo thời gian tính toán còn lại. Hướng thứ hai là bổ sung cơ chế phối hợp tấn công theo đội hình, thay cho việc các agent

tấn công Eagle một cách độc lập như hiện tại, nhằm tăng hiệu quả và tính hấp dẫn của gameplay. Hướng thứ ba là mở rộng hỗ trợ bản đồ động hoàn toàn, trong đó agent có thể phá hủy tường và thay đổi đồ thị bản đồ trong thời gian thực, đòi hỏi thuật toán MAPF có khả năng tái hoạch định toàn cục. Hướng cuối cùng là triển khai WebGL với PIBT server đặt trên cloud thay vì localhost, mở ra khả năng chạy backtest phân tán và demo trực tuyến không cần cài đặt.

TÀI LIỆU THAM KHẢO

- [1] R. Stern et al., “Multi-agent pathfinding: Definitions, variants, and benchmarks,” in *Proceedings of the 12th International Symposium on Combinatorial Search (SoCS)*, 2019, pp. 151–158.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [4] K. Okumura, M. Machida, X. Défago, and Y. Tamura, “Priority inheritance with backtracking for iterative multi-agent path finding,” in *Artificial Intelligence*, vol. 310, Elsevier, 2022, p. 103 752.
- [5] B. De Wilde, A. W. Ter Mors, and C. Witteveen, “Cooperative pathfinding,” in *Proceedings of the 5th International Conference on Agents and Artificial Intelligence (ICAART)*, vol. 2, 2013, pp. 186–194.
- [6] N. Sturtevant, *MovingAI MAPF benchmark*, 2019. Accessed: Jun. 24, 2026. [Online]. Available: <https://movingai.com/benchmarks/mapf.html>
- [7] N. R. Sturtevant, “Benchmarks for grid-based pathfinding,” in *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, 2012, pp. 144–148.
- [8] Unity Technologies, *Unity engine 6 documentation*, 2024. Accessed: Jun. 24, 2026. [Online]. Available: <https://docs.unity3d.com/6000.0/Documentation/Manual/>
- [9] J. Li, Z. Chen, D. Harabor, N. R. Sturtevant, and S. Koenig, “Anytime multi-agent path finding via large neighborhood search,” in *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 4127–4135.

PHỤ LỤC

A. KẾT QUẢ BACKTEST CHI TIẾT

Phụ lục này trình bày kết quả thực nghiệm backtest chi tiết theo từng bản đồ và thuật toán, dưới dạng trung bình kèm độ lệch chuẩn trên sáu lần lặp ($n = 6$). Các số liệu này bổ sung cho bảng tổng hợp trong Chương 4, phục vụ kiểm chứng độ ổn định của từng thuật toán. Cột thời gian tính theo giây, cột replan là tổng số lần tái hoạch định, cột shot là tổng số phát bắn trung bình.

A.1 Kết quả môi trường tĩnh

Bảng A.1 liệt kê kết quả chi tiết trong môi trường tĩnh.

Bảng A.1: Kết quả backtest chi tiết – môi trường tĩnh ($n = 6$)

Bản đồ	Thuật toán	Thời gian (s)	Replan	Shot TB
Alpha32	A*	$35,1 \pm 0,0$	159 ± 0	59182
Alpha32	PIBT C#	$32,3 \pm 0,4$	137 ± 1	60254
Alpha32	PIBT C++/TCP	$35,5 \pm 0,5$	137 ± 1	60672
Mansion	A*	$98,2 \pm 0,6$	672 ± 4	47810
Mansion	PIBT C#	$103,3 \pm 5,6$	3408 ± 1013	36968
Mansion	PIBT C++/TCP	$102,7 \pm 3,0$	3408 ± 1013	62612
Chantry	A*	$130,8 \pm 0,7$	936 ± 3	53402
Chantry	PIBT C#	$127,5 \pm 0,6$	3514 ± 261	36023
Chantry	PIBT C++/TCP	$134,4 \pm 1,2$	3514 ± 261	47211
Gallows	A*	$112,4 \pm 0,4$	788 ± 3	48593
Gallows	PIBT C#	$121,9 \pm 0,5$	4769 ± 234	25329
Gallows	PIBT C++/TCP	$115,8 \pm 2,8$	4769 ± 234	57208
Maze128	A*	$180,0 \pm 0,0$	1439 ± 1	302
Maze128	PIBT C#	$180,0 \pm 0,0$	1440 ± 59	11261
Maze128	PIBT C++/TCP	$180,0 \pm 0,0$	1440 ± 59	0

Độ lệch chuẩn nhỏ trên hầu hết tổ hợp cho thấy kết quả ổn định qua các lần lặp. Riêng bản đồ Mansion với PIBT có độ lệch chuẩn replan lớn (khoảng 1013), phản ánh tính ngẫu nhiên cao của quá trình phân giải xung đột trong hành lang hẹp.

A.2 Kết quả môi trường có chương ngại động

Bảng A.2 liệt kê kết quả chi tiết khi bật chương ngại động.

So với môi trường tĩnh, số lần replan tăng trên mọi tổ hợp, xác nhận cả ba thuật toán đều phản ứng với thay đổi môi trường bằng cách tái hoạch định nhiều hơn.

Bảng A.2: Kết quả backtest chi tiết – môi trường có chương ngại động ($n = 6$)

Bản đồ	Thuật toán	Thời gian (s)	Replan	Shot TB
Alpha32	A*	$40,4 \pm 0,1$	238 ± 0	65101
Alpha32	PIBT C#	$37,2 \pm 0,5$	178 ± 1	66280
Alpha32	PIBT C++/TCP	$40,8 \pm 0,6$	178 ± 1	66740
Mansion	A*	$112,9 \pm 0,7$	1007 ± 6	52592
Mansion	PIBT C#	$118,8 \pm 6,4$	4430 ± 1316	40664
Mansion	PIBT C++/TCP	$118,2 \pm 3,4$	4430 ± 1316	68873
Chantry	A*	$150,4 \pm 0,8$	1403 ± 5	58743
Chantry	PIBT C#	$146,6 \pm 0,6$	4569 ± 340	39625
Chantry	PIBT C++/TCP	$154,6 \pm 1,4$	4569 ± 340	51932
Gallows	A*	$129,3 \pm 0,5$	1182 ± 5	53452
Gallows	PIBT C#	$140,2 \pm 0,5$	6201 ± 304	27862
Gallows	PIBT C++/TCP	$133,2 \pm 3,2$	6201 ± 304	62929
Maze128	A*	$180,0 \pm 0,0$	2158 ± 2	332
Maze128	PIBT C#	$180,0 \pm 0,0$	1871 ± 76	12387
Maze128	PIBT C++/TCP	$180,0 \pm 0,0$	1871 ± 76	0

B. ĐẶC TẢ USE CASE BỔ SUNG

Phụ lục này đặc tả chi tiết hai use case bổ sung không trình bày trong Chương 2: chơi mạng LAN nhiều người (UC2) và xem kết quả backtest (UC5).

B.1 Đặc tả UC2 – Chơi mạng LAN nhiều người

Bảng B.1: Đặc tả UC2 – Chơi mạng LAN

Tên use case	Chơi mạng LAN nhiều người
Tác nhân	Người chơi chủ phòng (host), người chơi tham gia (client)
Tiền điều kiện	Các máy cùng mạng LAN; một máy tạo phòng, các máy khác tham gia
Luồng chính	1. Host tạo phòng, hệ thống khởi tạo máy chủ Fish-Net và hiển thị mã/địa chỉ phòng. 2. Các client nhập địa chỉ và tham gia phòng. 3. Mỗi người chơi chọn xe tăng và bấm sẵn sàng (ready). 4. Khi tất cả sẵn sàng, host bấm bắt đầu; hệ thống đồng bộ bản đồ và spawn xe tăng cho mọi người chơi. 5. Số agent AI được tính theo công thức 6 agent mỗi người chơi; các agent tấn công Eagle Base chung. 6. Trạng thái xe tăng, đạn và Eagle được đồng bộ giữa các máy theo thời gian thực.
Luồng phát sinh	2a. Client nhập sai địa chỉ → báo lỗi kết nối, quay lại màn hình nhập. 4a. Một client mất kết nối → host nhận thông báo và tiếp tục ván chơi với số người còn lại.
Hậu điều kiện	Ván chơi kết thúc đồng bộ trên mọi máy; hiển thị kết quả chung

B.2 Đặc tả UC5 – Xem kết quả backtest

Bảng B.2: Đặc tả UC5 – Xem kết quả backtest

Tên use case	Xem kết quả backtest
Tác nhân	Người dùng (nhà nghiên cứu)
Tiền điều kiện	Đã chạy ít nhất một đợt backtest và sinh tệp kết quả
Luồng chính	1. Người dùng mở thư mục BacktestResults/. 2. Mở tệp backtest_summary_*.csv để xem kết quả tổng hợp theo từng run. 3. Mở tệp backtest_chart_*.html để xem biểu đồ trực quan trên trình duyệt. 4. Mở tệp backtest_agents_*.csv để xem chi tiết chỉ số từng agent.
Luồng phát sinh	3a. Nếu môi trường thiếu Python/matplotlib, tệp PNG không sinh được nhưng HTML vẫn hiển thị bảng số liệu.
Hậu điều kiện	Người dùng nắm được kết quả định lượng của đợt thực nghiệm